



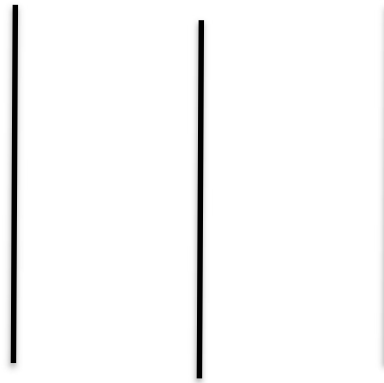
Universitat de Girona



IFROS 2024-2026

Autonomous System

Lab-4: RRT & RRT STAR (RRT*)



Delivered by:

Pravin Oli (u1999699)

Gebrecherkos Gebreslassie (u1999542)

Supervisor:

Narcís Palomeras Rovira

Sebastian Realpe Rua

Date of Submission:

09/12/2024

Table of Contents	
1. Introduction	3
1.1. Sampling Based Motion Planner	3
1.2. RRT Algorithm Concept	3
1.3. RRT* Algorithm Concept	4
2. Implementation	4
2.1 Part A Map Loading	4
2.2 Part B RRT Implementation	4
2.3 Part C RRT* Implementation	5
3. Output & Result	6
3.1 Map with start and goal	6
3.2 RRT and RRT* Results	6
4.Problems Encountered	12
5. Conclusion	13

1. Introduction

1.1. Sampling Based Motion Planner

Sampling based motion planner (SBMP) consists a class of algorithms which rely on random samples points in configuration space (C Space), rather than mapping and discretizing for finding a feasible path to move from start to goal. SBMP first explores the unknown environment by generating random samples (q_{rand}), then it connects those sample points by verifying that the paths between them are collision-free, iterating until goal is reached.

Traditional grid-based and search-based motion planners have a drawback that they cannot be implemented on high-dimensional complex environments, but this approach is useful for solving path planning problems in high-dimensional complex environments.

The main principle behind SBMP is:

- a. **Sampling:** Randomly sample points from the configuration space (C-space).
- b. **Collision Checking:** Checks sampled points and connections between them are free from obstacles.
- c. **Pathfinding:** Connects valid points to build a tree and path by linking near node to new node.
- d. **Iterative Exploration:** Continue sampling and connecting until a solution is found or iteration is maximum.

1.2. RRT Algorithm Concept

1. **Initialize the Tree:** Start the tree with the initial configuration q_{start} .
2. **Sample a Random Point:** Select a random configuration q_{rand} from the configuration space.
3. **Find the Nearest Node:** Locate the nearest vertex q_{near} in the current tree to q_{rand} .
4. **Steer Towards q_{rand} :** Generate a new configuration q_{new} by moving from q_{near} toward q_{rand} , constrained by a step size Δt .
5. **Collision Check:** Verify that the trajectory from q_{near} to q_{new} is collision-free.
6. **Add q_{new} :** If q_{new} is collision-free, add it to the tree as a new node with an edge connecting it to q_{near} .
7. **Repeat:** Continue the process from step 2 until either:
 - o q_{new} reaches q_{goal} or
 - o A maximum number of iterations is exceeded.
8. **Return the Path:** If q_{goal} is reached, retrieve the path from q_{start} to q_{goal} .

Path Smoothing:

RRT produces path with unnecessary sample points which cause longer path so to make path shorter unnecessary nodes are filtered out and path is reconstructed.

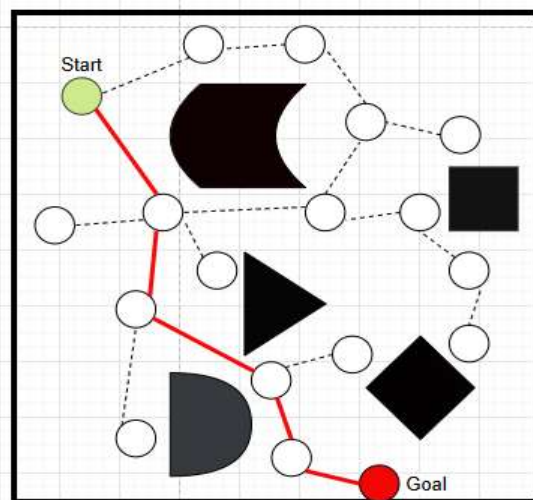


Figure1:RRT concept

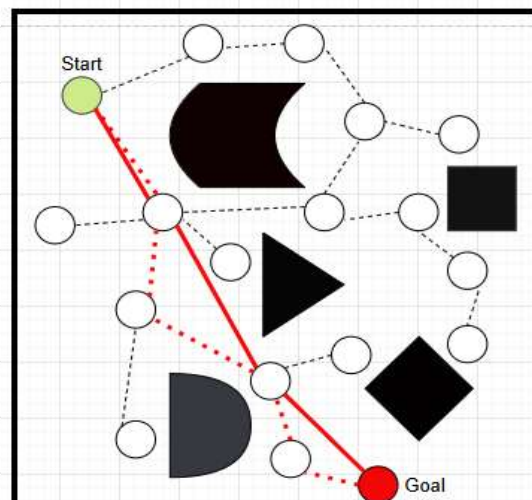


Figure 2 Path Smoothing concept

1.3.RRT* Algorithm Concept

1. **Initialize** tree with the starting configuration q_{start} .
2. **Sample** random configuration q_{rand} in the configuration space.
3. **Find Nearest Vertex** q_{near} in the current tree to q_{rand} .
4. **Move (Steer) Toward q_{rand}** from q_{near} by a step size Δt , to produce a new node q_{new} .
5. **Collision Check**: Ensure the path between q_{near} and q_{new} is free of obstacles.
6. **If q_{new} is Collision-Free**: Add q_{new} to the tree, find all nearby nodes within a radius r from q_{new} , Choose the best parent for q_{new} by minimizing cost. Rewire the tree by checking if any nearby nodes can be connected to q_{new} at a lower cost.
7. **Repeat**: Continue sampling and expanding the tree until q_{new} reaches q_{goal} or the maximum number of iterations is reached.
8. **Return Path**: Once q_{goal} is reached, return the optimized path from q_{start} to q_{goal} .

2. Implementation

This lab is implemented in three parts. The notebook ipynb file is organized in similar way dividing the parts:

1. Part A Map Loading
2. Part B RRT Implementation
3. Part C RRT* Implementation

2.1 Part A Map Loading

Code Organization in ipynb notebook

Import Libraries:

(Numpy, matplotlib, PIL, random)

Map Configuration:

(Map0, Map1, Map2, Map3 with start&goal)

User Input for map selection:

Enter number between 0 and 3

Load and Display:

(Plot selected map with start & goal)

2.2 Part B RRT Implementation

Code Organization in ipynb notebook

Define Point Class:

(2D points with x and y coordinates)

Define helper functions:

(fill_path, is_segment_free, smooth path, nearest_vertex, compute_path distance)

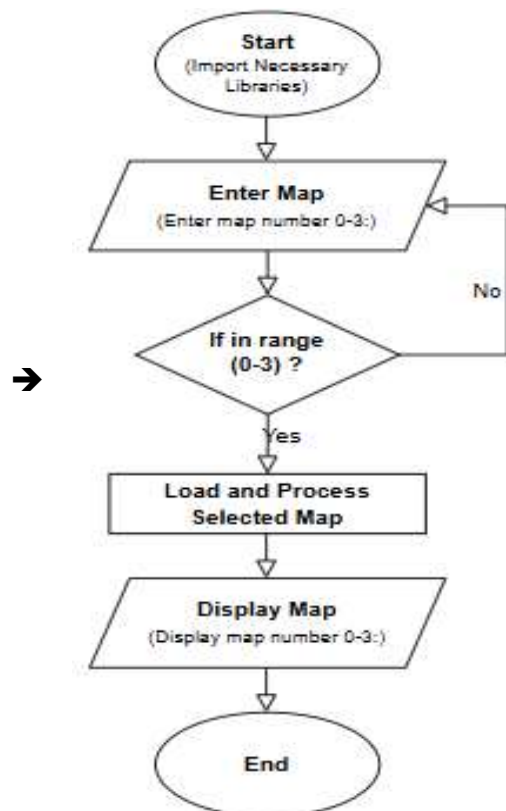


Figure-3: Flowchart for part A

RRT Function:

(rrt_with_iterations(grid_map, qstart, goal, delta_q, max_iter, p))

User input for maximum iterations (K):

(Enter the maximum number of iterations (K) for RRT:)

RRT Execution:

(Initialize parameters and runs RRT function)

Visualization RRT path:

(Plots the grid map, the RRT tree, and the final path).

Path Smoothing:

(Minimizes path length)

Visualization smooth path:

(Plots tree, original path, smoothened path)

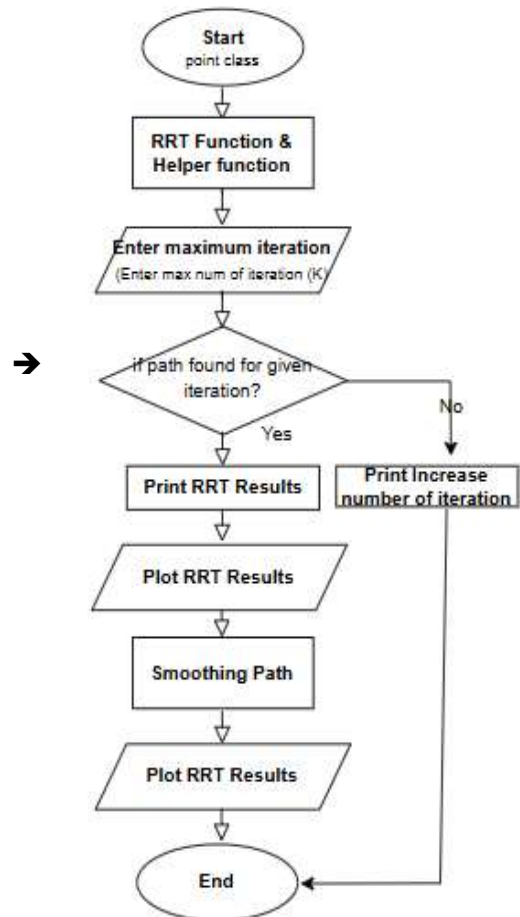


Figure-4: Flowchart for part B

2.3 Part C RRT* Implementation

Code Organization in ipynb notebook

Define Node Class:

(2D points with x and y coordinates)

Define helper functions:

(is_free, steer, get_near_node, collision free)

RRT* Function:

(Find near node, rewire, extract full tree path)

User input for maximum iterations (K):

(Enter the maximum number of iterations (K) for RRT*:)

Visualization RRT* path for goal reach iteration:

(Plots the grid map, wiring tree, and the final path).

Visualization RRT* path for maximum iteration:

(Plots the grid map, wiring tree, and the final path).

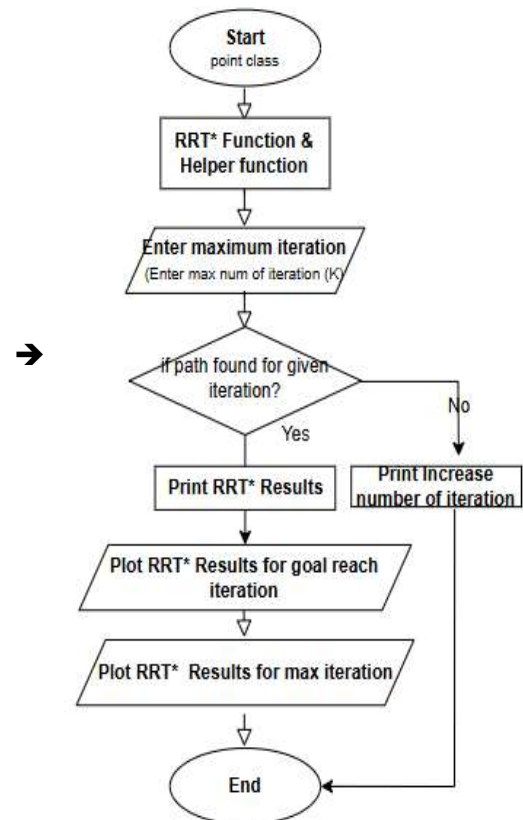


Figure-5: Flowchart for part C

3. Output & Result

3.1 Map with start and goal

Select a map:

0: map0.png

1: map1.png

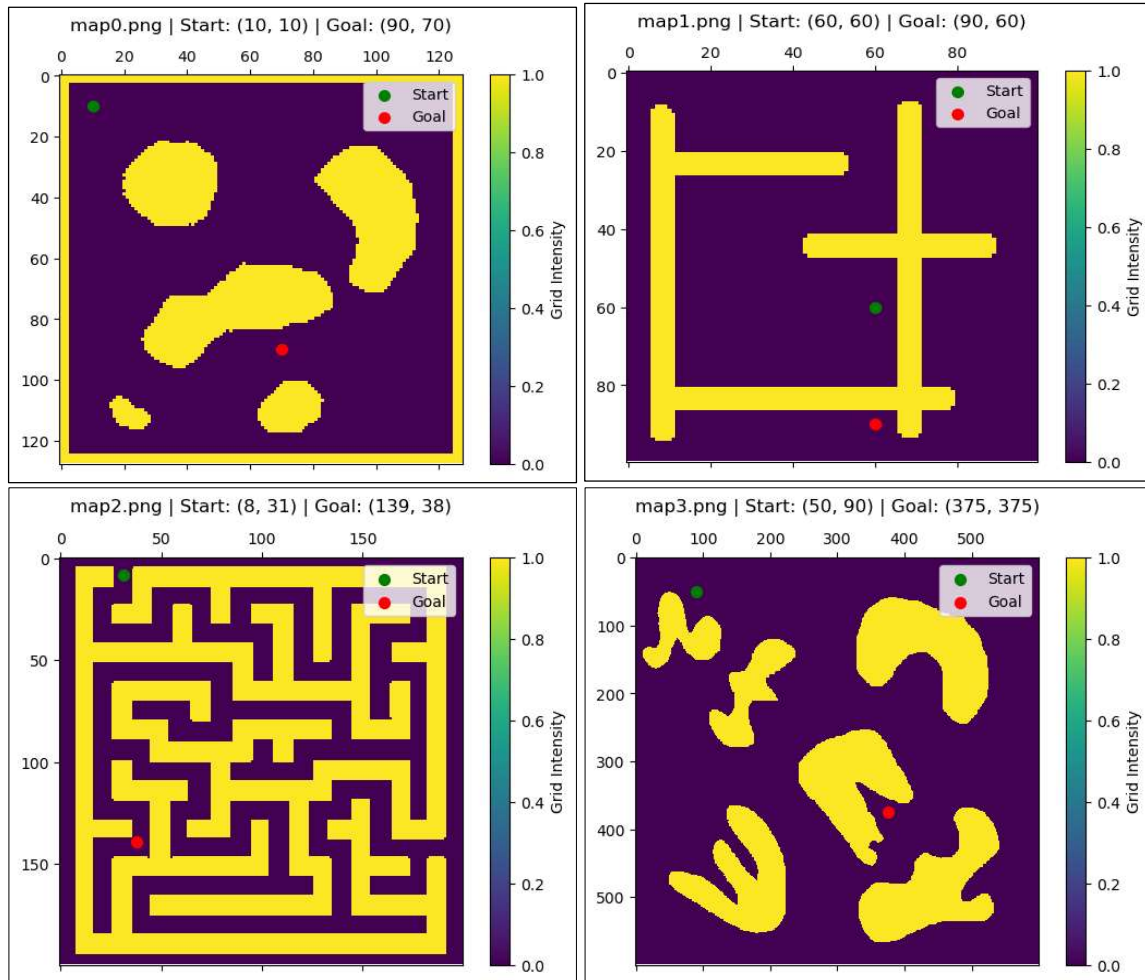
2: map2.png

3: map3.png

For example:

you have entered: 1

Selected map with start and goal point is: {'filename': 'map1.png', 'start': (60, 60), 'goal': (90, 60)}



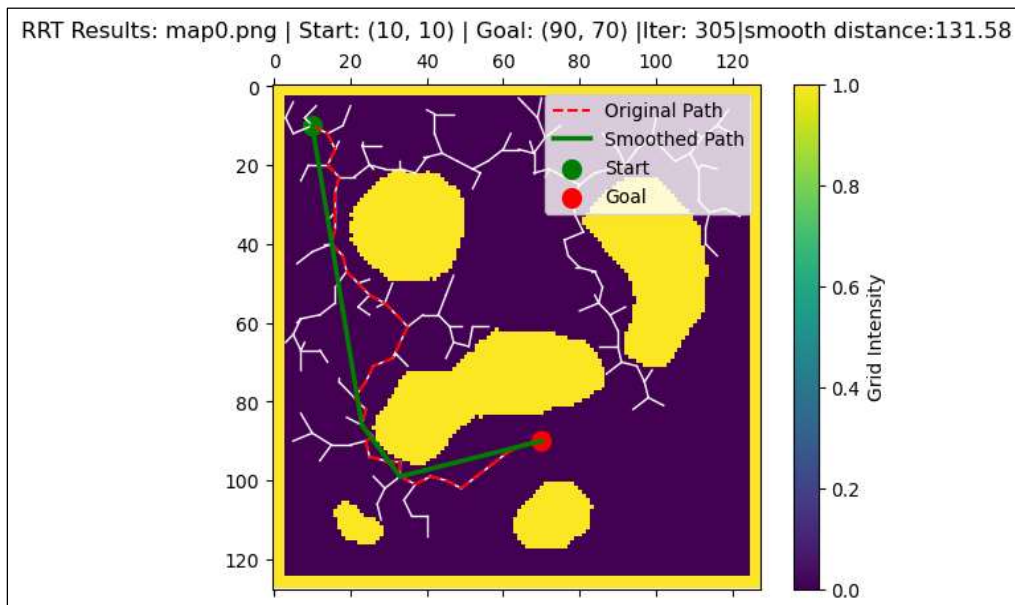
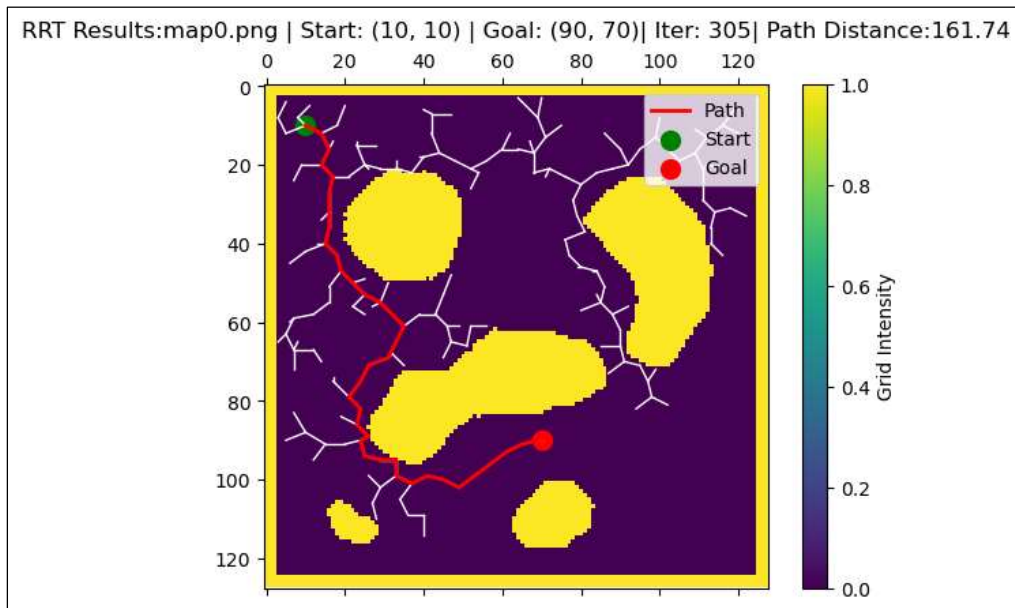
3.2 RRT and RRT* Results

Table :1 Results

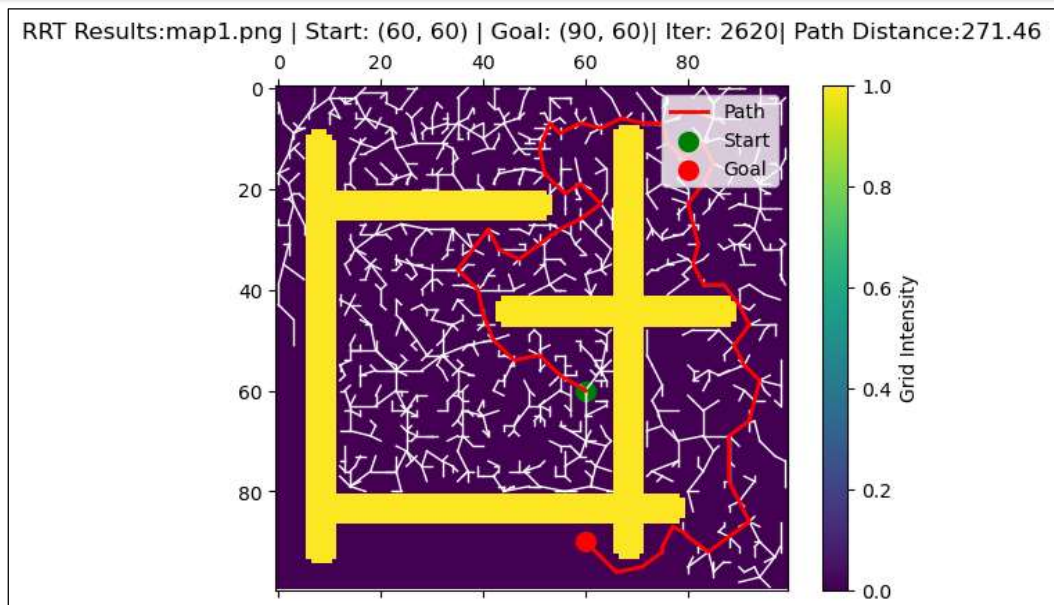
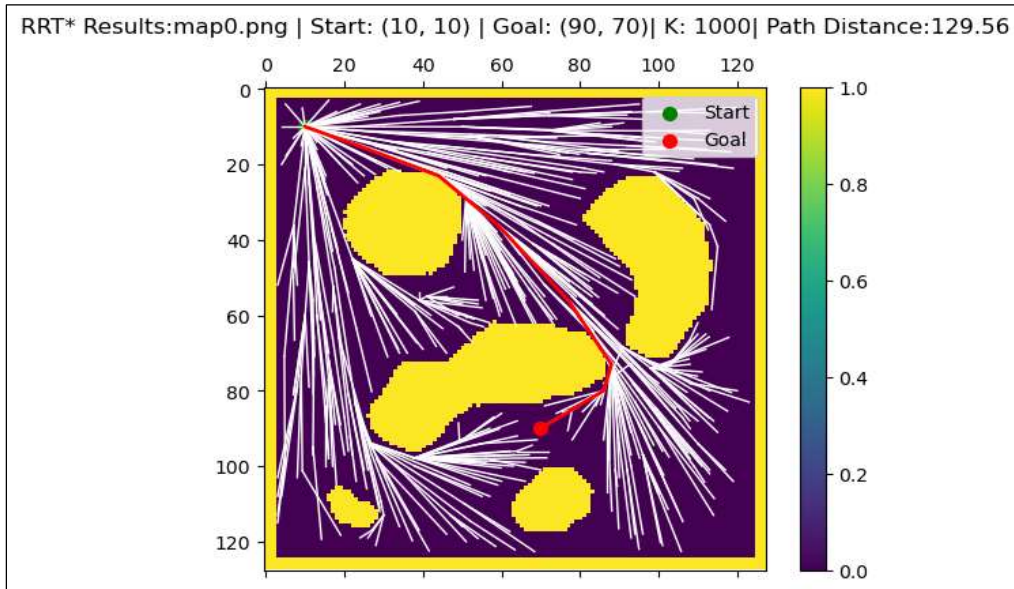
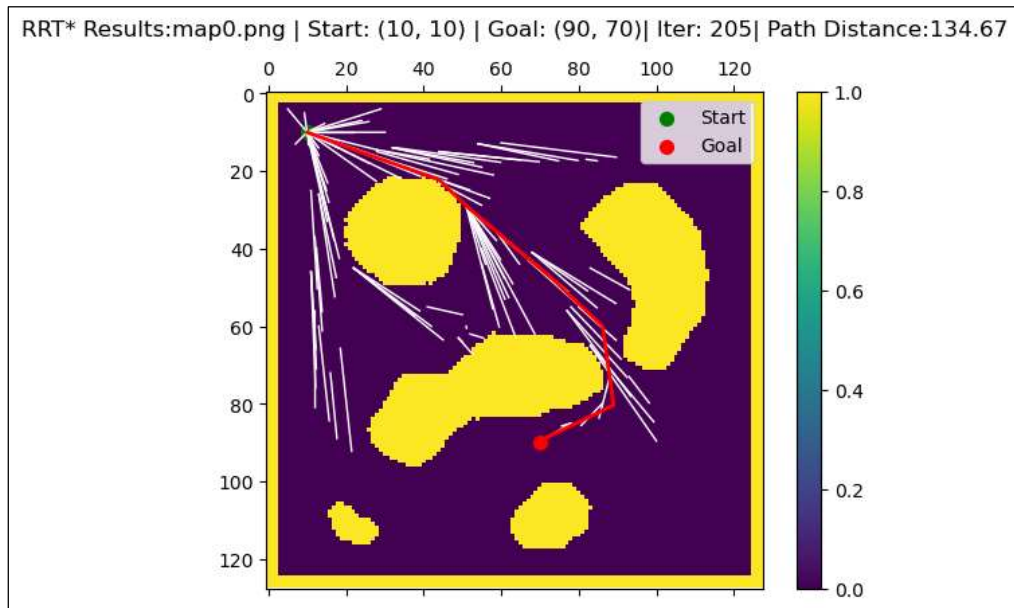
Maps	Start	Goal	RRT Result			RRT Star Result			
			Goal reach iter.	Path distance	Smooth distance	Goal reach iter.	Path distance	Maximum iter.	Path Distance
Map_0	10,10	90,70	305	161.74	131.58	205	134.67	1000	129.56
Map_1	60,60	90,60	2620	271.46	206.86	1975	185.8	3000	184.54
Map_2	8,31	139,38	33252	953.99	843.84	33310	552.59	40000	547.25
Map_3	50,90	375,375	930	767.89	546.74	620	529.17	1500	526.17

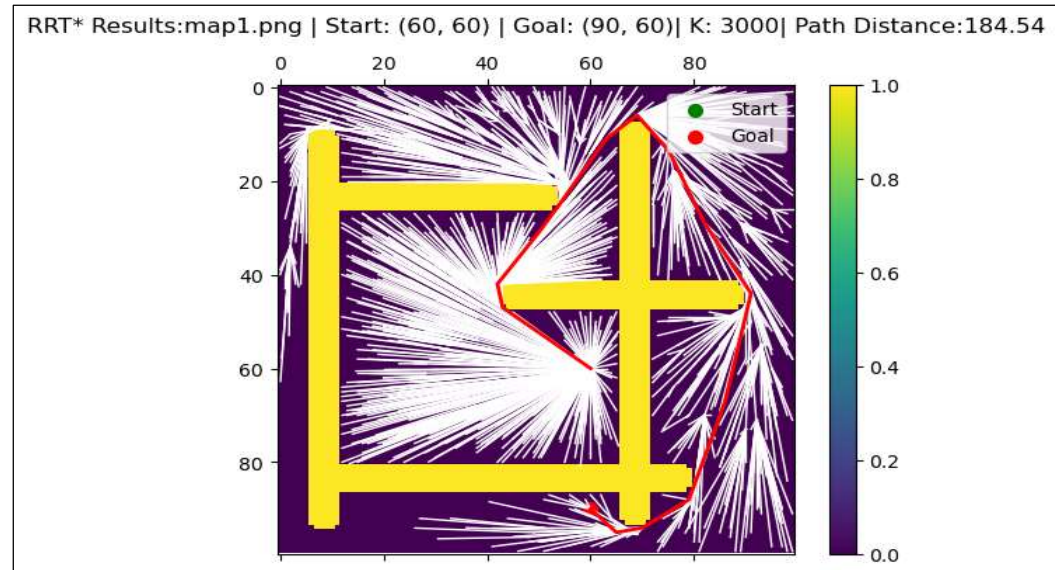
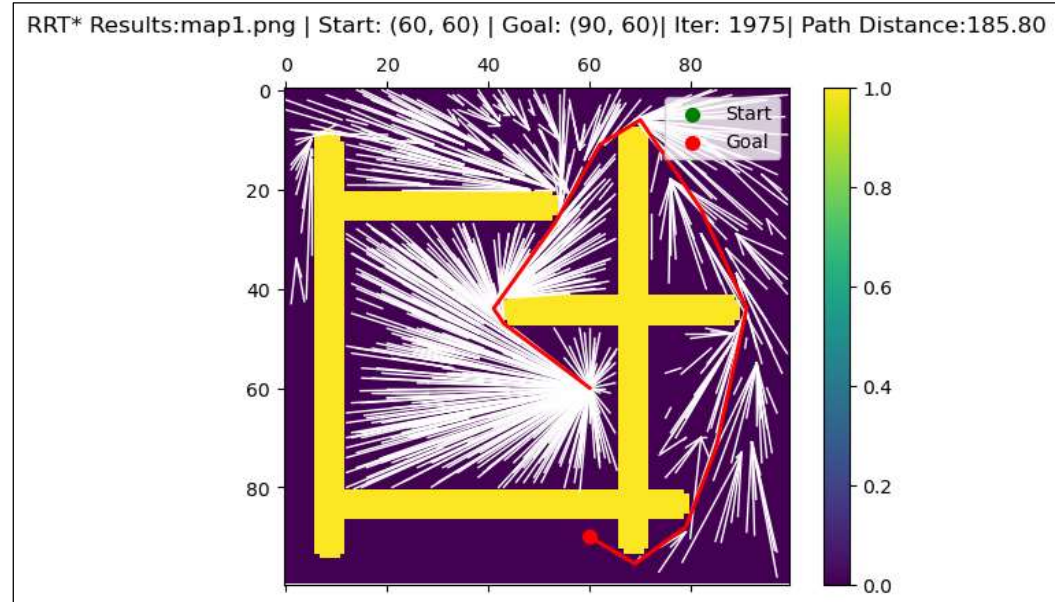
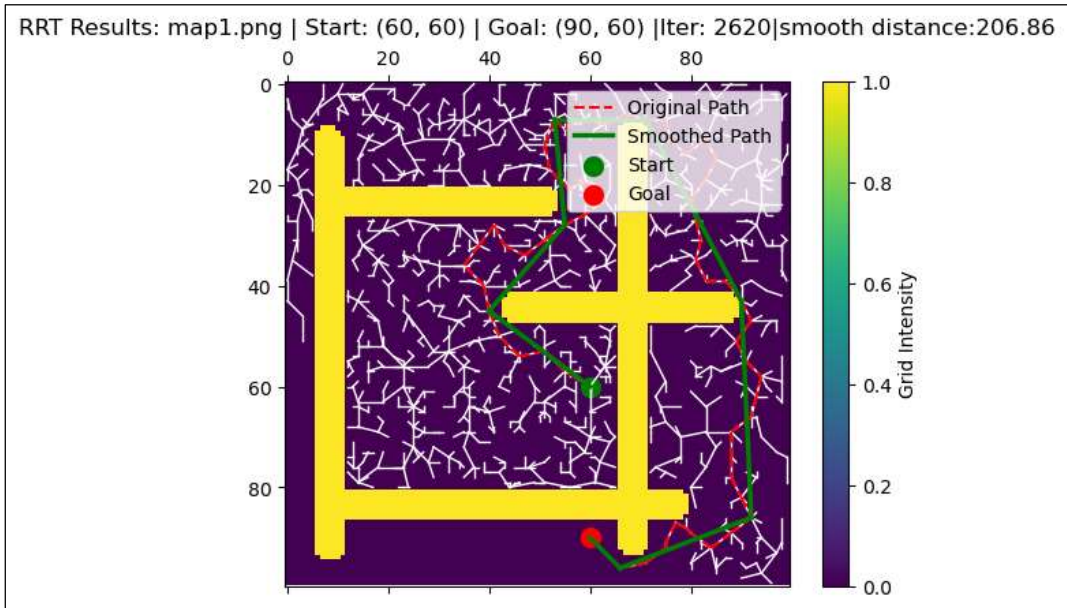
Observation and Analysis from Table 1:

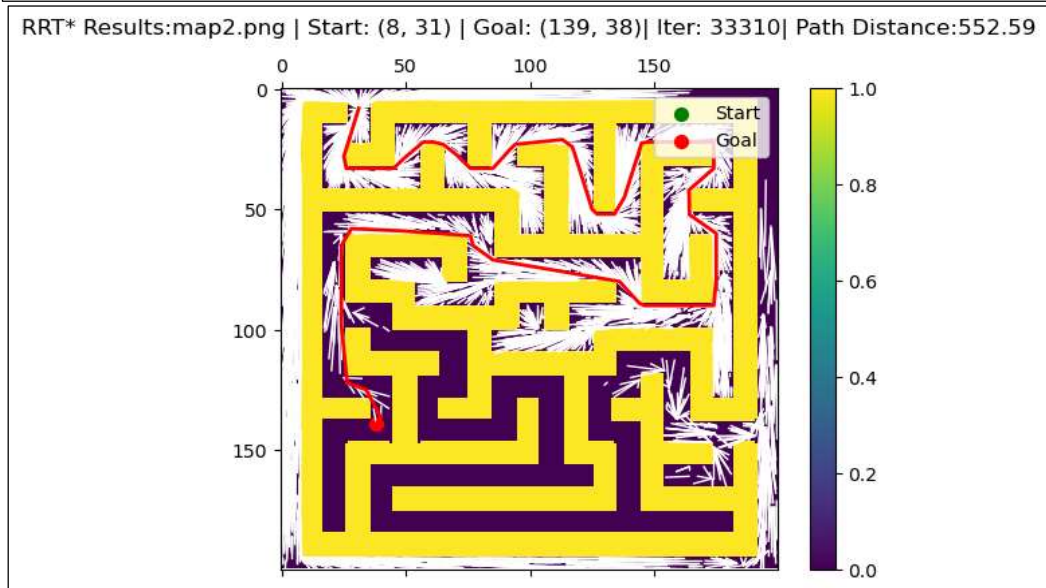
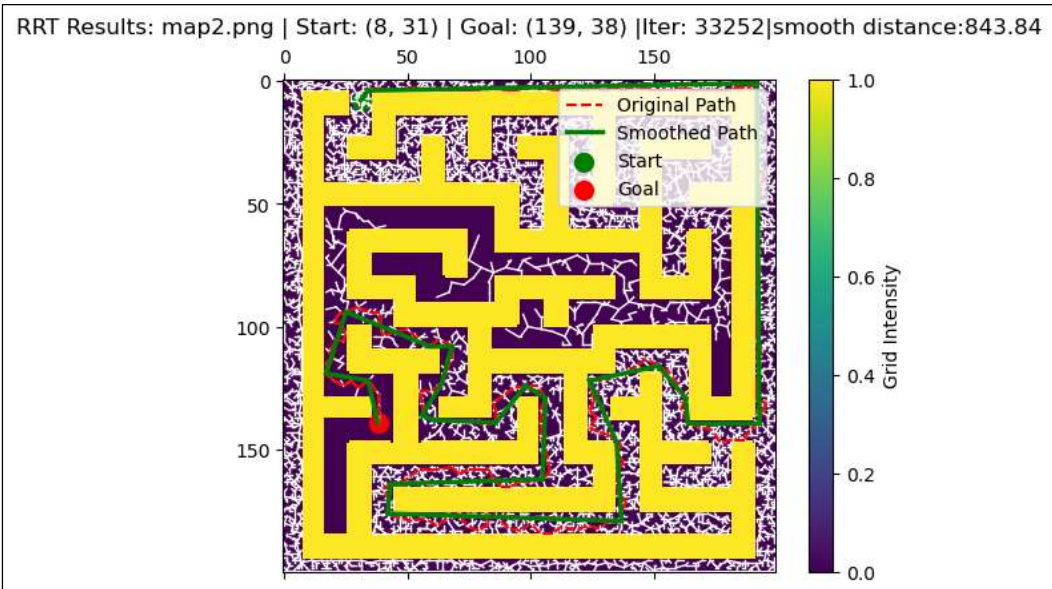
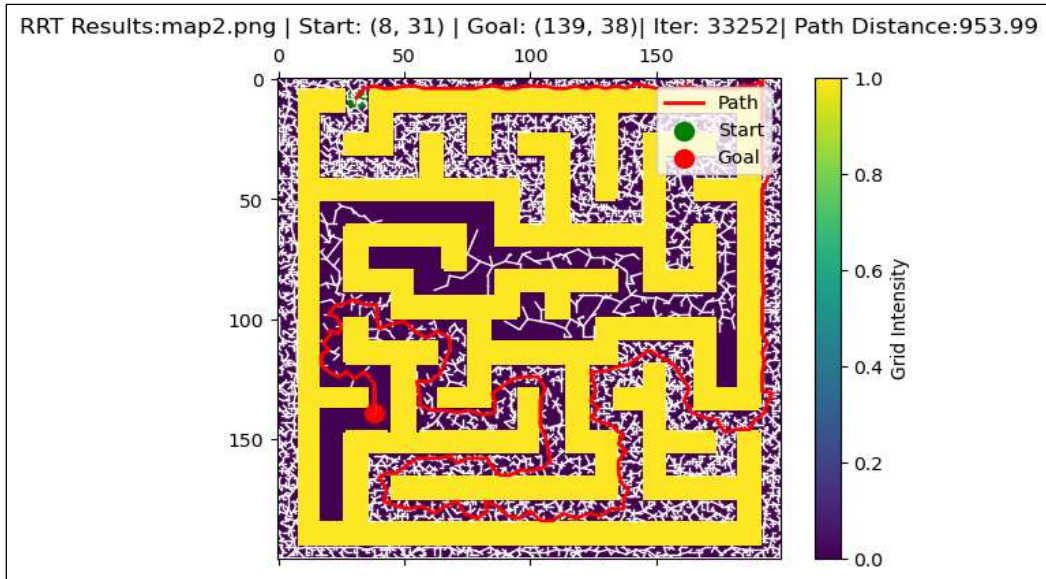
- Comparing RRT with Smoothing, Path distance shorter for smoothing. (Smoothing is better for short path)
- Comparing RRT with RRT* Goal reach iteration is less for RRT* and path distance is also shorter (winner is RRT* in terms of less iteration and shorter path distance)
- For Goal reach and Maximum iteration running for RRT*, The more the iteration we run the shorter path we get. *(But after some more iteration there is no significant reduction in path distance.)*

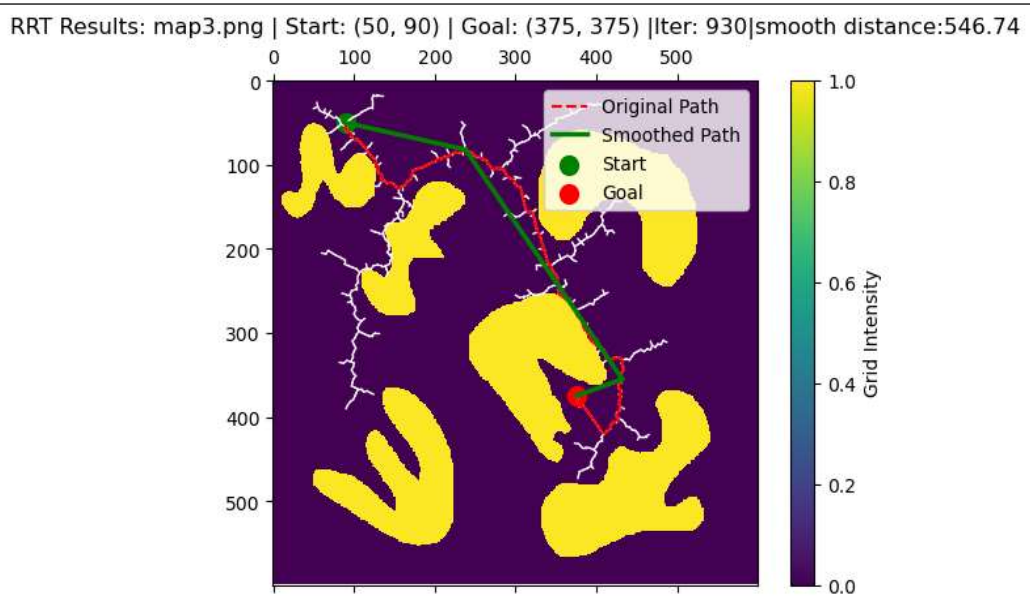
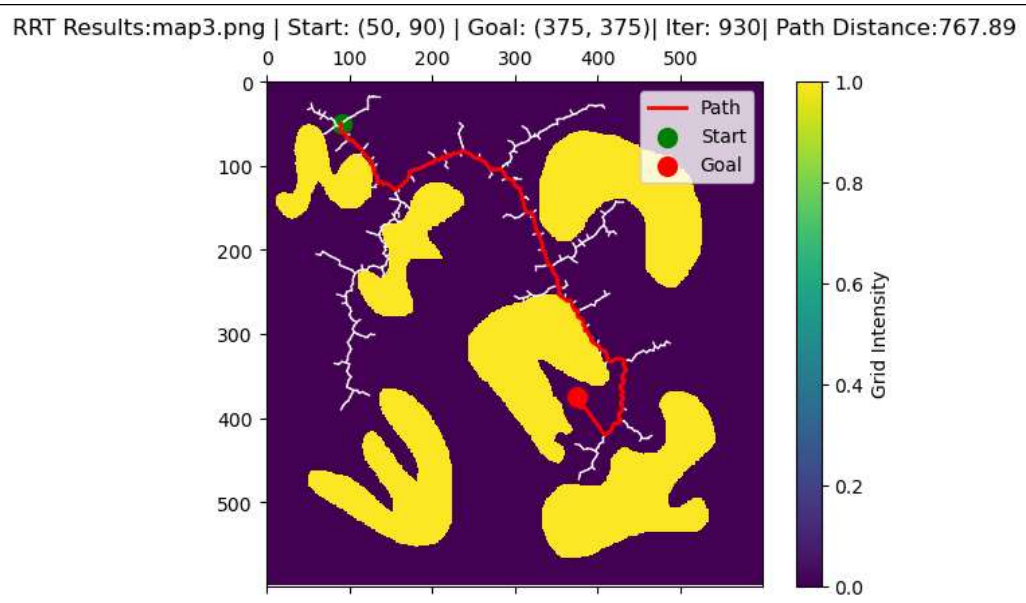
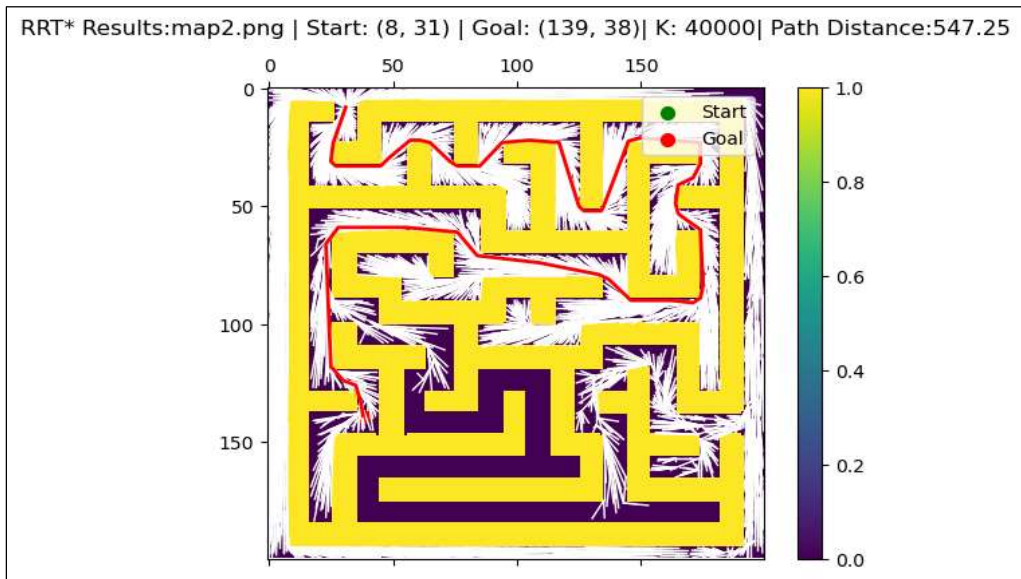


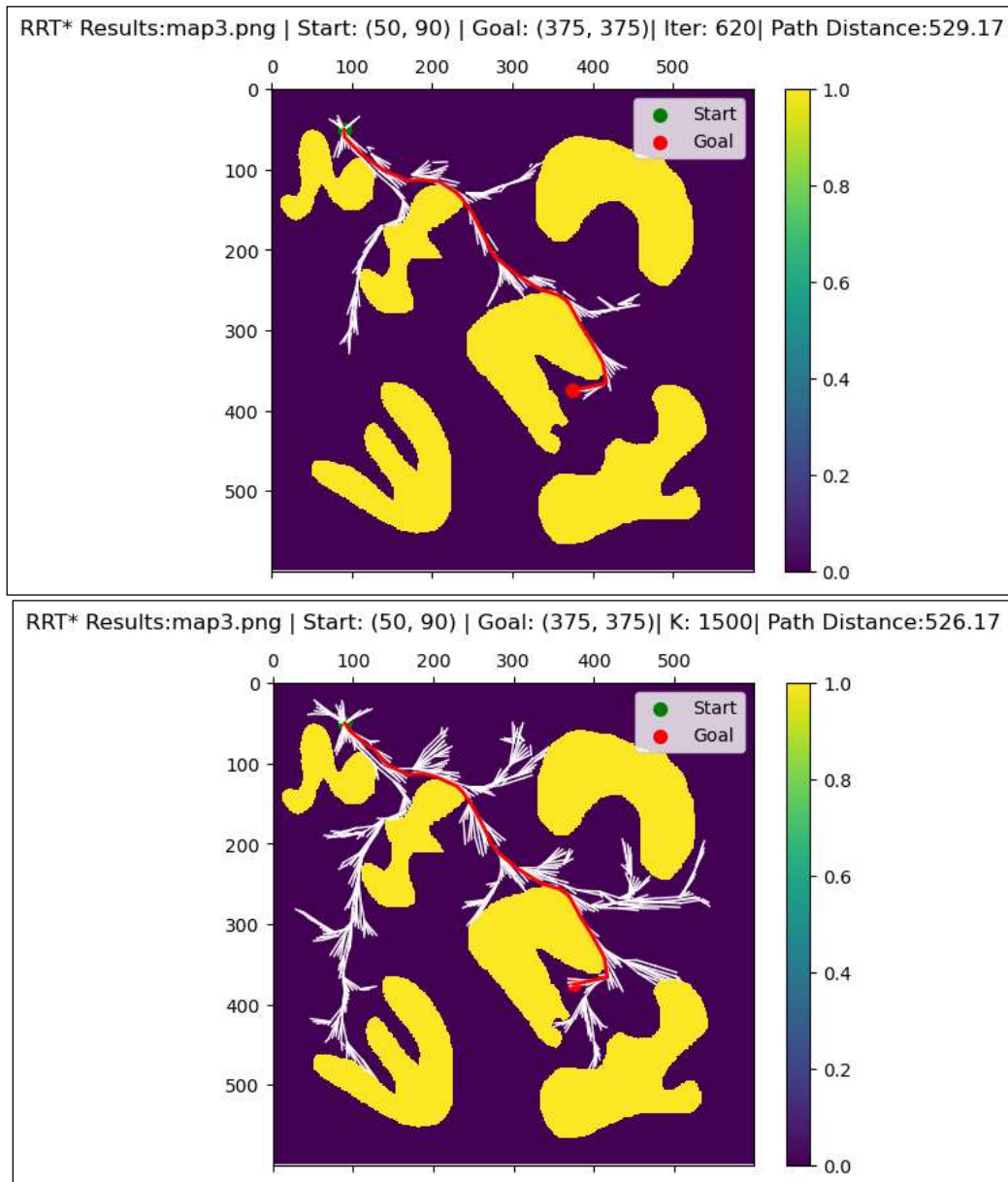
Lab 4 – RRT & RRT Star- (Pravin Oli, Gebrecherkos Gebreslassie)











4.Problems Encountered

Problem 1: Handling Multiple Maps

- **Challenge:** Generating outputs for all maps required a flexible, user-friendly interface.
- **Solution:**
 - a. Developed a user-defined input interface within the main function.
 - b. Allowed users to seamlessly select maps, ensuring smooth execution and output generation for all scenarios.

Problem 2 working with complex map and number of iteration setup.

- **Challenge:** Although map0 and map3 are simple but map2 is complex. So, I was unable to get result by fixing same number of iterations for all maps because complex max needs very large iteration comparing with simple map. (Map 2 is very complex I have to run between 35,00 to 40,000 iteration)
- **Solution:**
 - a. Created a user defined interface to provide number of iterations as per needed.
 - b. When iteration is not sufficient it shows message to increase iteration in output

Problem 3: Problem in visualizing tree at goal reach iteration and maximum iteration

- **Challenge:** I was using break on *after_goal_reach_iteration* so I was unable to get trees to maximum iteration for RRT*.
- **Solution:**
 - a. I removed break after goal reach iteration and extracted max iteration tree using lists extract full tree.

5. Conclusion

The lab exercises effectively demonstrated the use of the RRT and RRT* algorithm for finding the path from start to goal in various grid-based scenarios. RRT explores the configuration space by iteratively sampling random points, steering toward them from the nearest tree node, and adding collision-free paths to build a tree until the goal is reached. RRT* extends this by optimizing the path, rewiring the tree to minimize path costs, and achieving shorter, more efficient path. Comparative results observed from Table 1, highlighted the benefits of path smoothing in RRT and the superior performance of RRT* in terms of goal reach iterations and shorter path distances. The challenges encountered, such as handling complex maps and visualizing trees at different iterations, were addressed through user-defined interfaces and iterative adjustments, making solutions flexible. Overall, the lab highlighted the strengths of Sampling based algorithm in solving diverse pathfinding challenges.